CEITEC

MUNI



# C9555: A survival guide to simulations

Timothée Rivel – Sept. 26, 2024

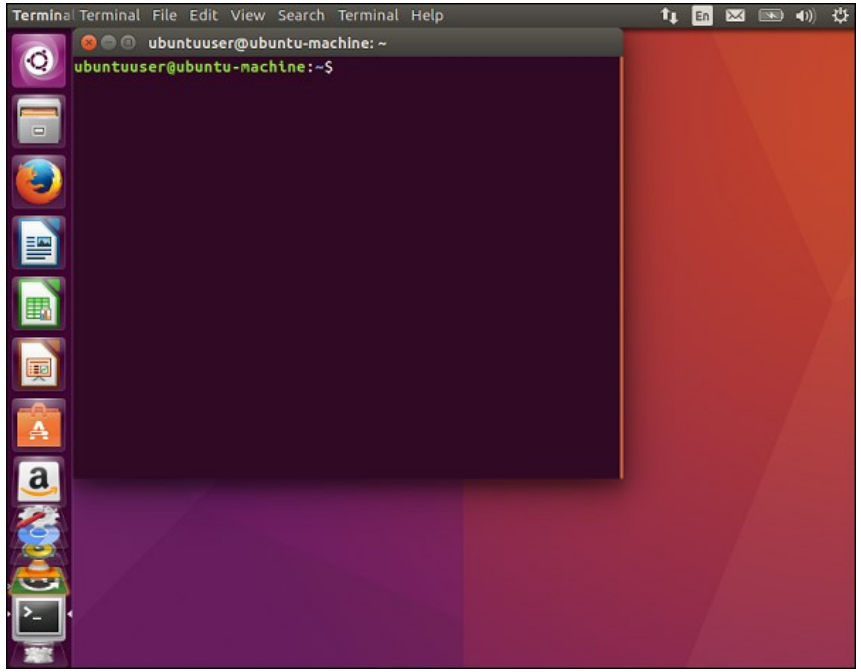
# Pulling in MD

- Today's plan
  - Potential energy surface
  - Biasing potentials
  - Restraints in GROMACS
  - Collective variable
  - Pulling Code in GROMACS

| TEACHING WEEK | DATE | LECTURER | TOPIC |
|---|---|---|---|
| 01 | 26.09 | TR | Introduction and visualization with VMD |
| 02 | 03.10 | WM | MD Theory and Lennard-Jones Phases |
| 03 | 10.10 | WM | Lennard Jones Polymers |
| 04 | 17.10 | DB | How to run MD simulations? |
| 05 | 24.10 | DB | Adding salt to our water |
| 06 | 31.10 | TR | Pulling in MD |
| 07 | 07.11 | TR | Enhanced sampling in MD |
| 08 | 14.11 | DB | Membranes in MD |
| 09 | 21.11 | IK | Sampling in MD |
| 10 | 28.11 | IK | Free energies (1/2) |
| 11 | 05.11 | IK | Free energies (2/2) |
| 12 | 12.12 | WM | Using Alpha Fold in MD |

# Getting ready

- **The support of the course is on Gitlab:**
  - Open a new terminal by pressing `CTRL+ALT+T`



# Getting ready

- **One way to download files (you can also use git a better way):**

```
$mv survival_guide_simulations_c9555/week05 ~/c9555/
$rm -rf survival_guide_simulations_c9555/
$git clone https://gitlab.com/denysbiriukov/survival_guide_simulations_c9555.git
$cd survival_guide_simulations_c9555/week06
```

- **Activate necessary modules and conda environment**

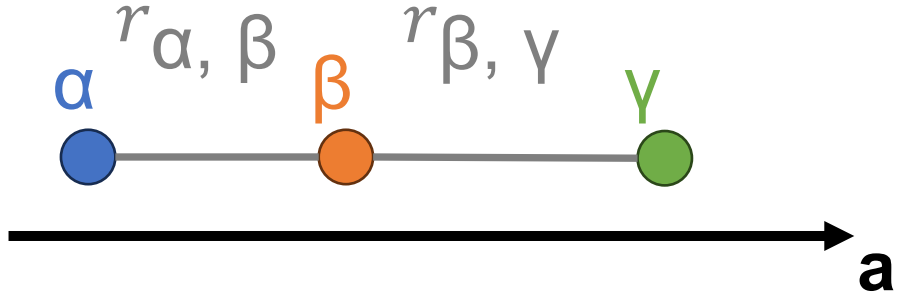
```
$bash ../infinity/infinity-temp.sh
$bash ../conda/conda-activate.sh
```

- **Starting Jupyter notebook:**

```
$bash ../conda/jupyter-start.sh
```

# Potential Energy Surface (PES)

- Let's make our universe (a tad) simpler
  - The universe is now **1D**
  - The only coordinate of this universe is called **a**
  - It is composed of **1 single molecule**
  - This molecule has **3 atoms**: α, β, γ

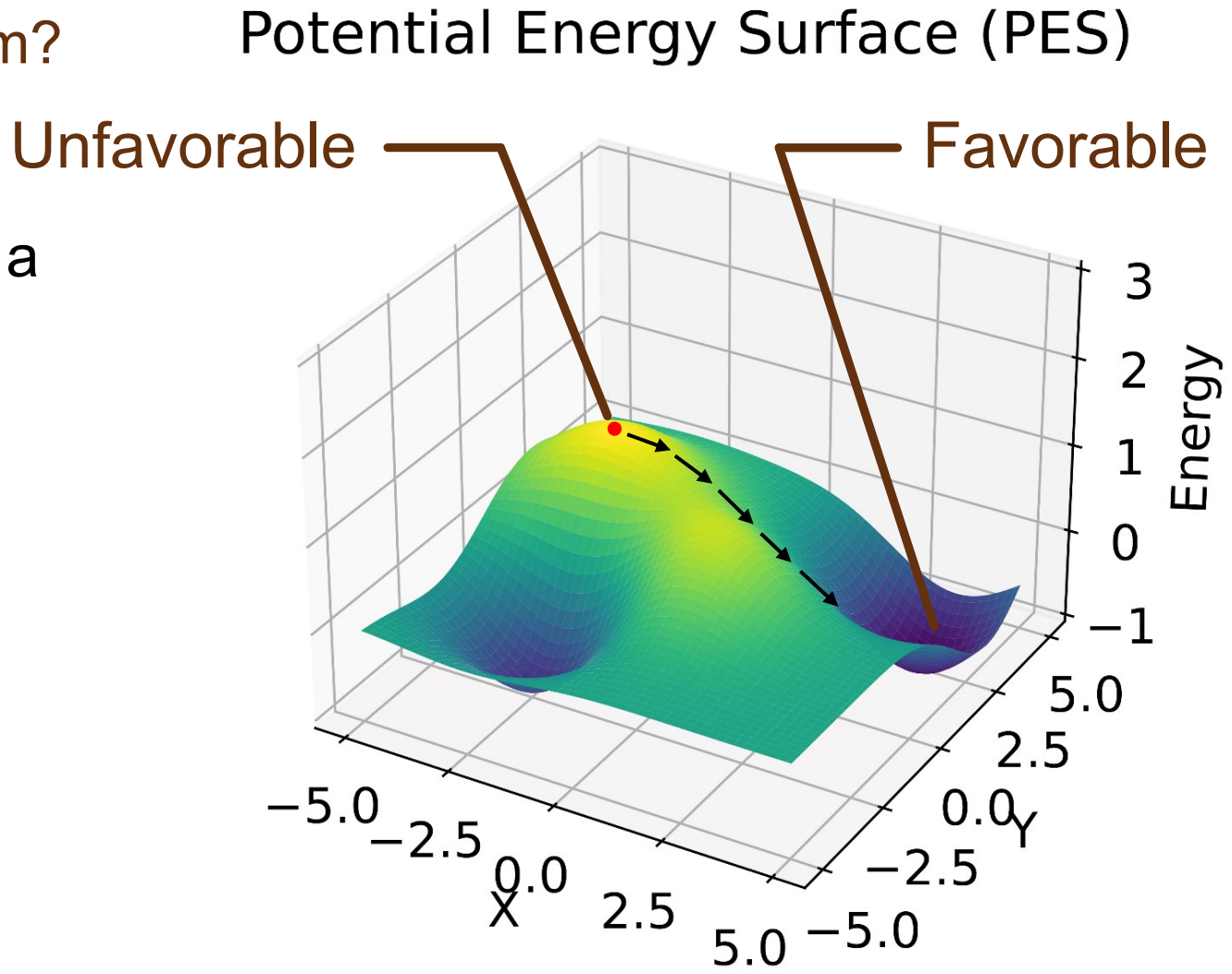



- Coordinates
  - α, β, γ positions are defined along **a**
  - Customary to use **internal coordinates** (invariance)
  - The distances $r_{\alpha,\beta}$ and $r_{\beta,\gamma}$ define all degrees of freedom

- For simplicity let's write
  - $X = r_{\alpha,\beta}$ and $Y = r_{\beta,\gamma}$

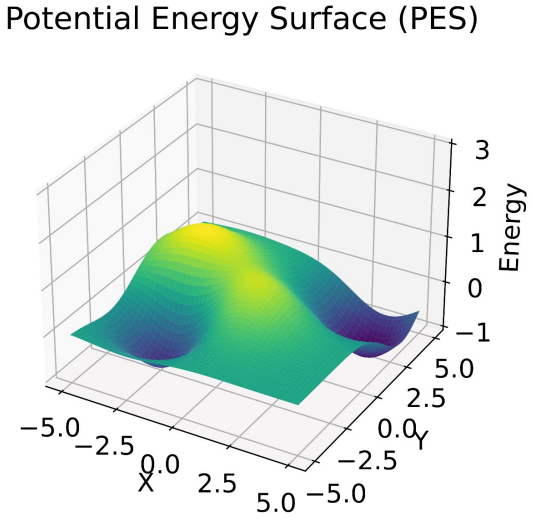
# Potential Energy Surface (PES)

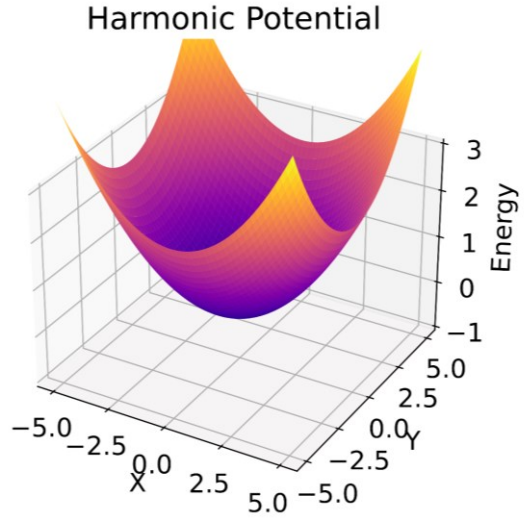
- What's the potential energy of a system?
  - Comes from intra- and inter-molecular interactions
  - Characterize relative probability of a configuration in absence of motion
- Let's plot the energy *V* of the system
  - *V* depends on X and Y
  - The value of the energy is **relative**
  - Each point is a **configuration**
  - High energy: **unfavorable**
  - Low energy: **favorable**
- Evolution of the system
  - $F(X,Y) = -\nabla V(X,Y)$

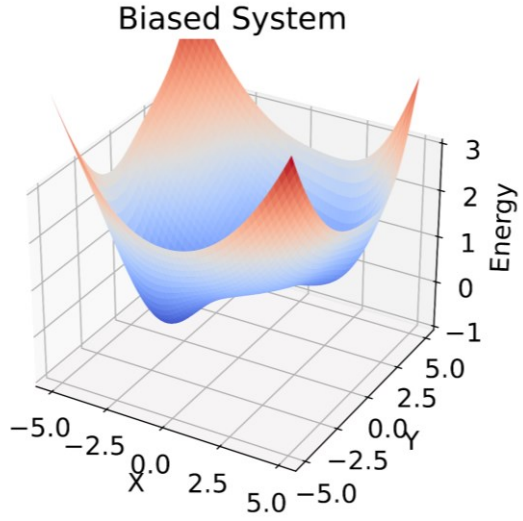


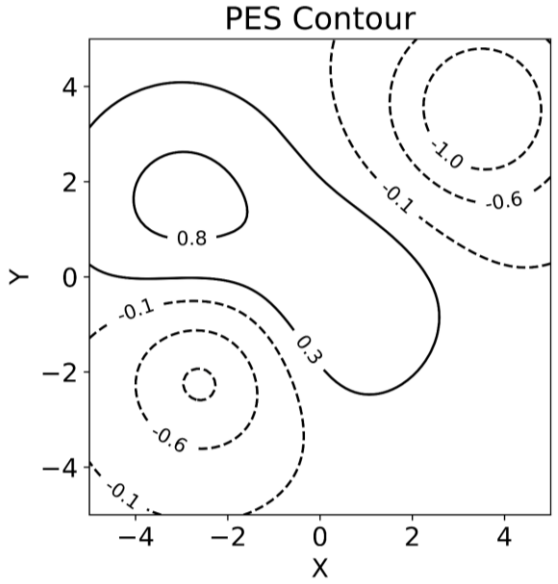
# Biasing potential

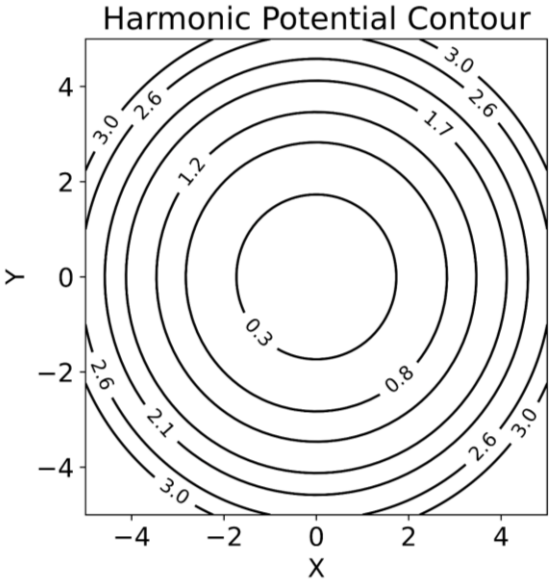
- Summing potentials
  - Let's use previous system
  - And add a harmonic potential
  - $V_{Tot}(X,Y) = V(X,Y) + V_H(X,Y)$
- Energy penalty
  - **Restrains** the system in a subset of the **configurational space**

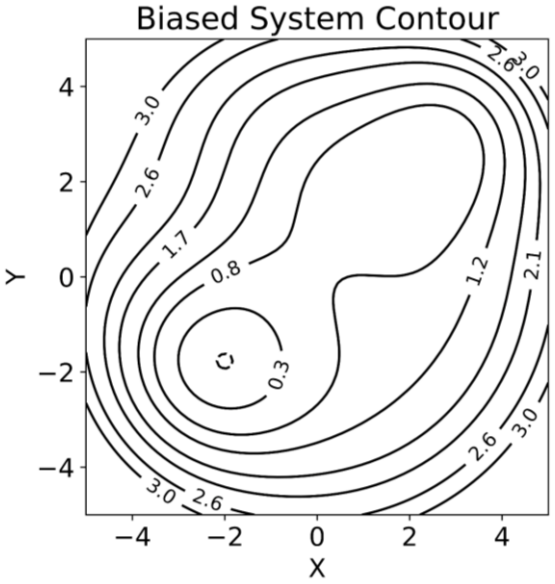


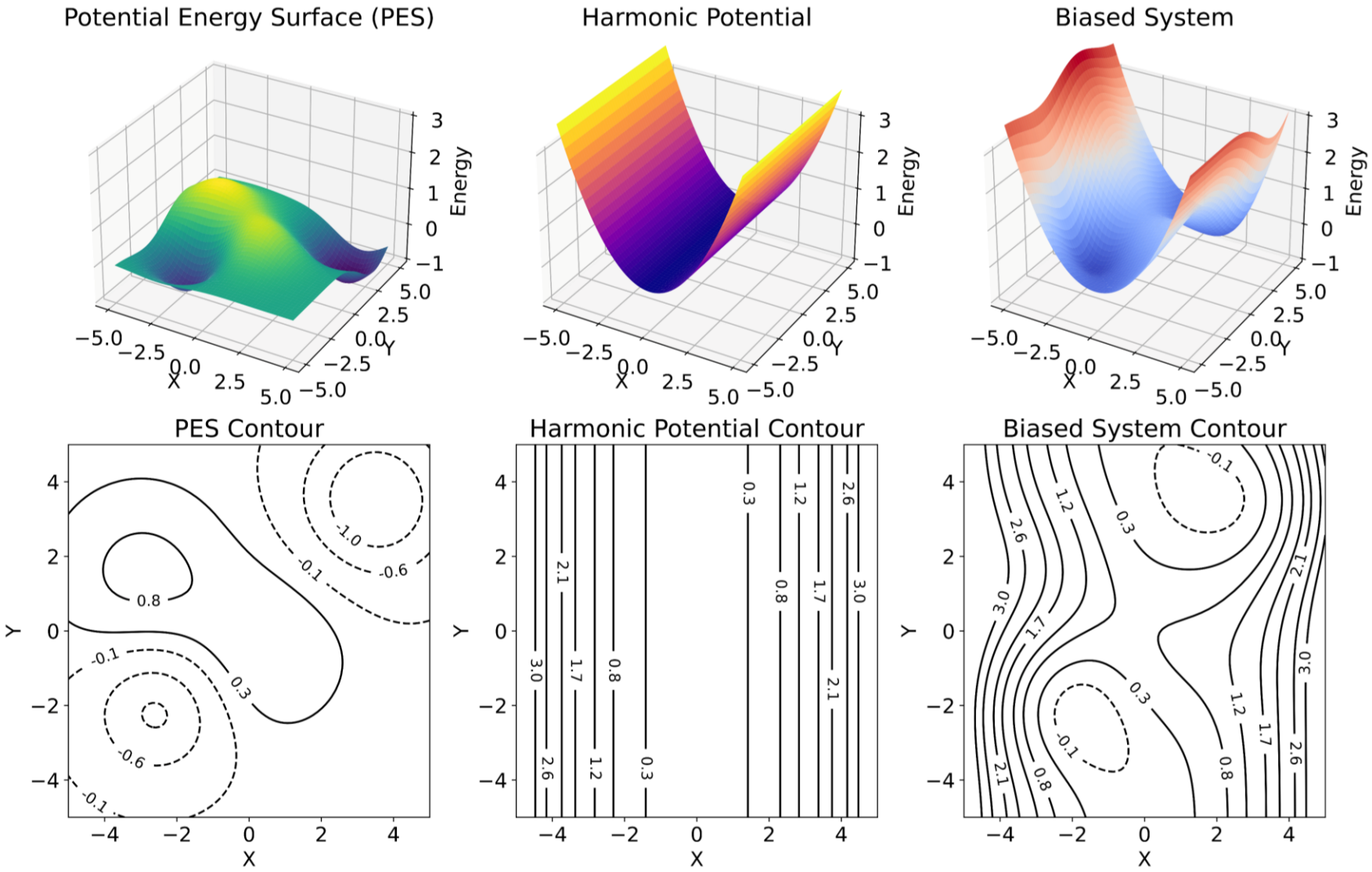



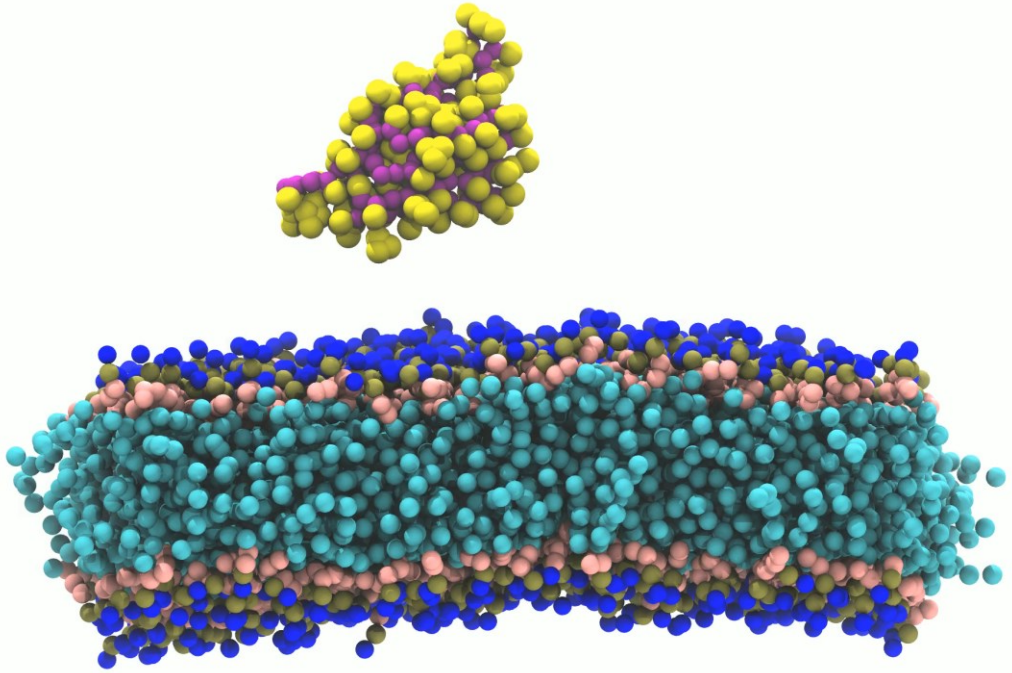
# Biasing potential

- Another example



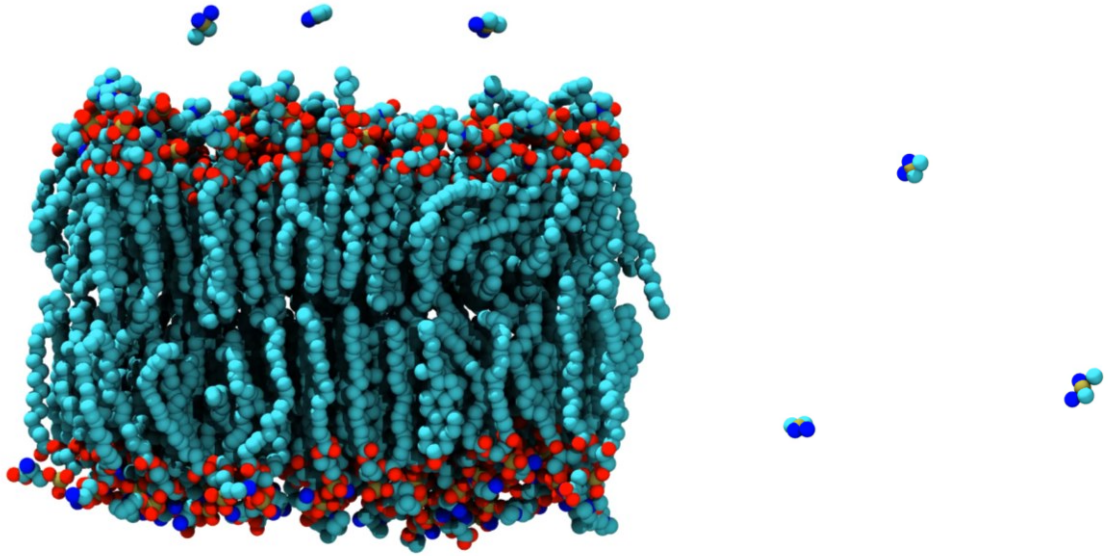
# Position restraints

- Applying biasing potentials with GROMACS
  - For equilibration purposes



**Backbone** of the protein is **restrained**
The protein does not adsorb, but **side chains** can **relax**

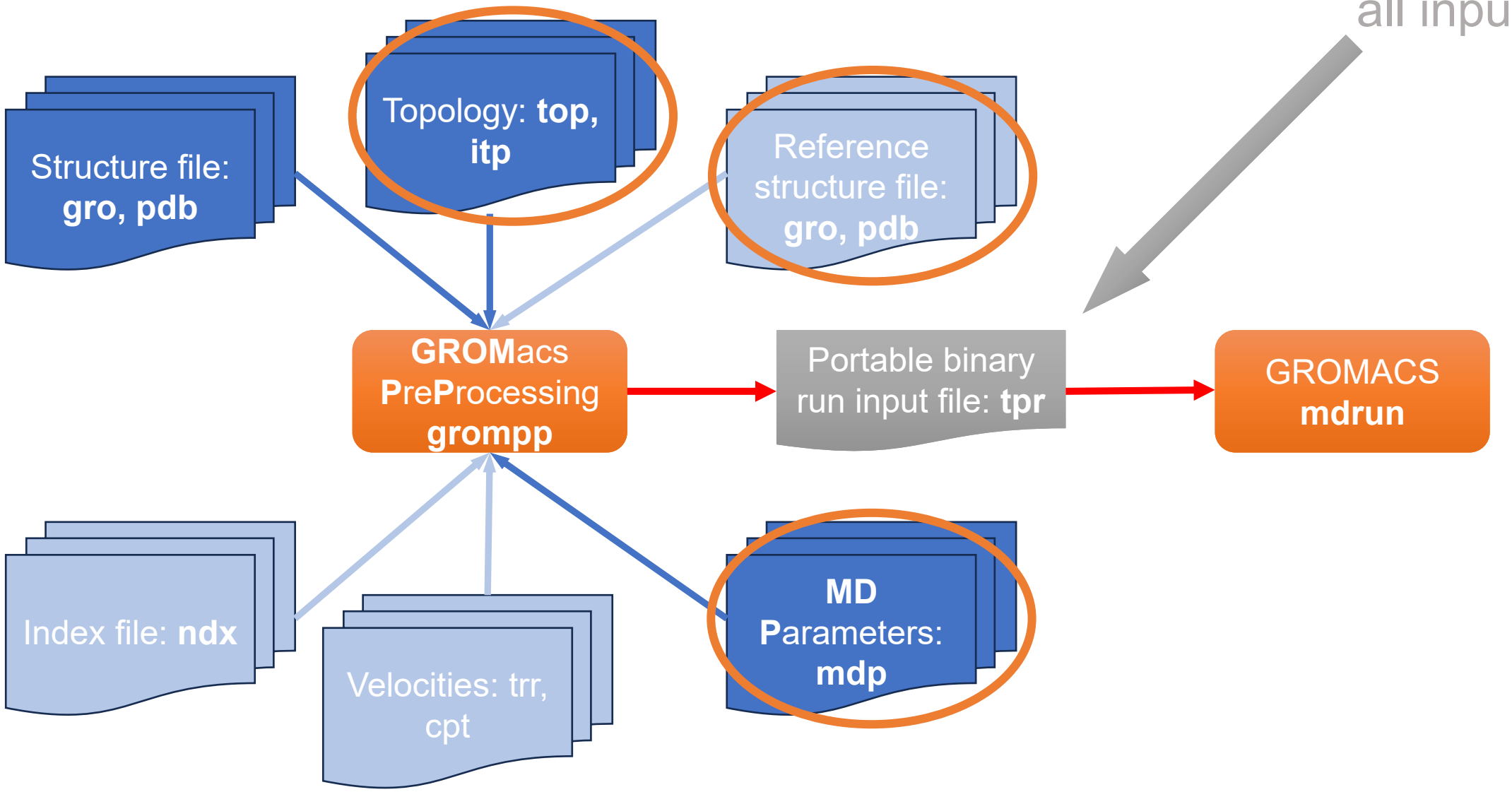
  - For specific setups



Side view Top view

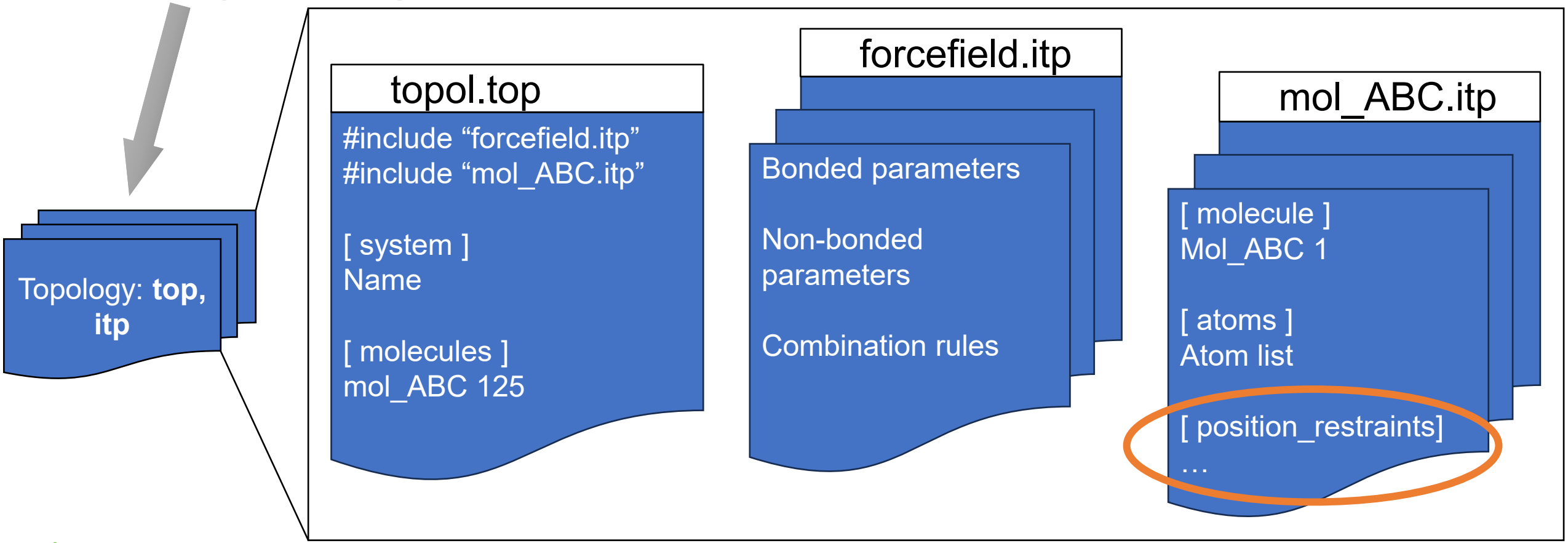
Cancer drugs do not interact with one another
1 simulation: multiple events

# How does GROMACS work?



# Setting up restraints – Topology files

- Based on sections called **directives**: [ abce_fghij ]
- **Modular**: #include
- Recognizes **logical statement**: #ifdef ABC… #else… #endif



# Setting up restraints – Topology files

- Based on sections called **directives**: [ abce_fghij ]
- **Modular**: #include
- Recognizes **logical statement**: #ifdef ABC… #else… #endif

➢ The directive [ position_restraints ]

- There can be multiple directive
- Applies a harmonic potential on the listed atoms:

$$V_{pr}(r_i) = \frac{1}{2} k_{pr} |r_i - R_i|^2$$

- Format:

```
atom_id function_id=1          kx    ky    kz
```

- Example:

```
12                      1      1000     0   500
37                      1         0     0   700
```

# Setting up restraints – Conditional statement

- Examples:

```
#ifdef POSRES
[ position_restraints ]
12                    1      1000     0   500
37                    1         0     0   700
#endif

#ifdef POSRES_K
[ position_restraints ]
12                    1  POSRES_K     0     0
37                    1  POSRES_K     0     0
#endif
```

- Good practice
- Variable should be defined in the mdp

# Setting up restraints – Conditional statement

- How to set the restraint in the mdp file:

```
#ifdef POSRES
[ position_restraints ]
12                  1       1000     0   500
37                  1          0     0   700
#endif
```

- mdp file:

```
define = -DPOSRES
```

# Setting up restraints – Conditional statement

- How to set the restraint in the mdp file:

```
#ifdef POSRES_K
[ position_restraints ]
12                    1 POSRES_K     0     0
37                    1 POSRES_K     0     0
#endif
```

- mdp file:

```
define = -DPOSRES_K=700
```

# Setting up restraints – flat-bottom restraints

- Restraints made to restrict some atoms to a portion of the simulation box:
  - Applies a harmonic potential on the listed atoms:

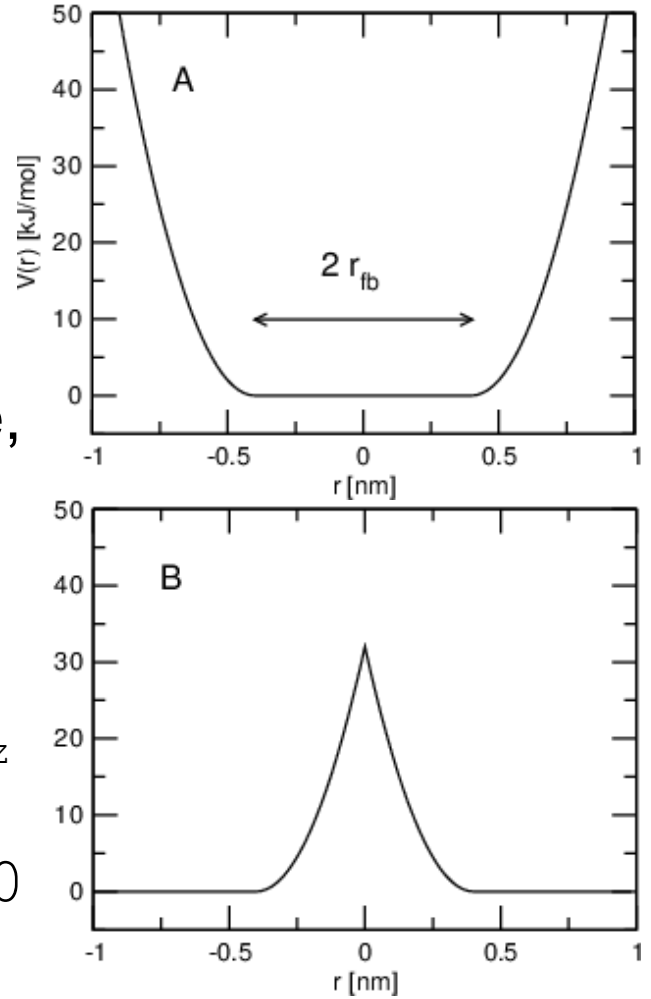
$$V_{fb}(r_i) = \frac{1}{2} k_{fb} \left[ d_g(r_i; R_i) - r_{fb} \right]^2 H\left[ d_g(r_i; R_i) - r_{fb} \right]$$

  - $d_g$ is a distance that depends on the geometry (layer, sphere, cylinder)
  - *H* is the Heaviside function
  - Format:

```
atom_id function_id=2   g     r_fb          k_x     k_y     k_z
```

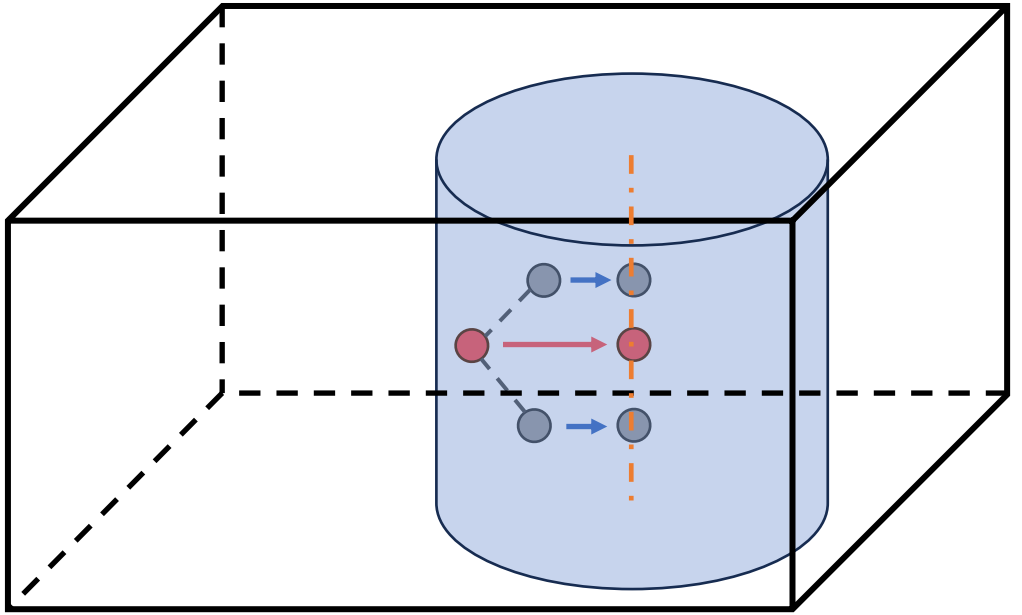
  - Example:

```
12                      2   8       2       1000      0     500
```



# Setting up restraints – Reference structure file

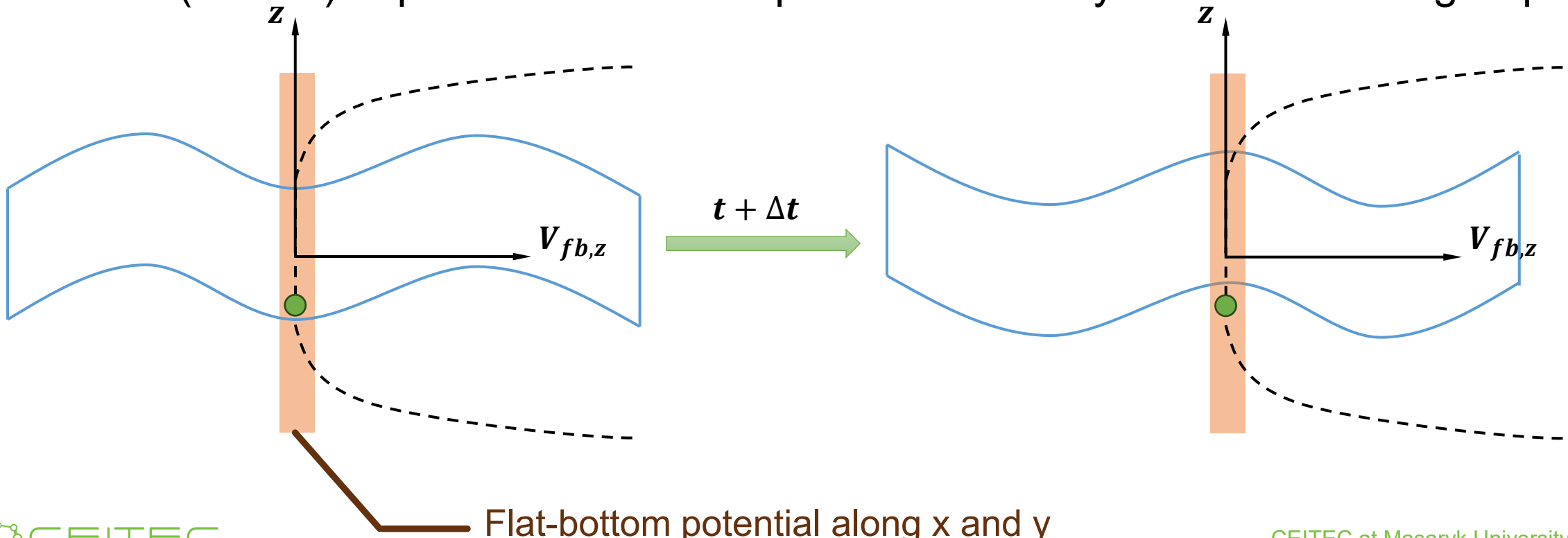
- Reference structure file for position restraints
  - The input structure file of grompp is generally the one used as a reference file
  - Good practice: if you run multiple simulations with position restraints, keep the same reference structure file all along



- Reference structure file for flat-bottom position restraints
  - Each restrained atom should be placed carefully as their position is used to define the origin of the flat-bottom region!
  - In other words: you will need to modify the initial gro/pdb file

# Collective variable

- Limitations of GROMACS restraints code
  - The biasing potential is applied to atoms of one molecule type
  - The center of the biasing potential is defined once at the beginning of the simulation
    - (almost) impossible to set this position relatively to other atomic groups



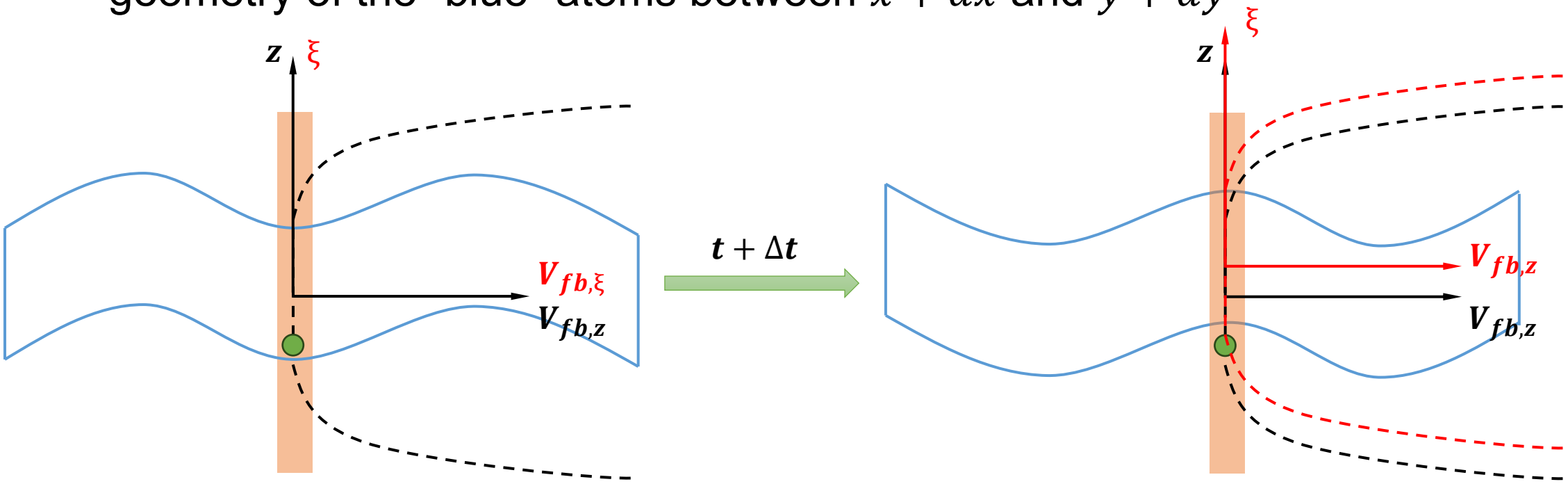
# Collective variable

## ➢ Definitions

- **Collective variables (CVs)** are **functions** of the atomic positions of some group(s) of atoms present in our system
- They generate a **curvilinear coordinate**, *i.e.* a path on the configuration space
- They should be:
  - **Continuous:** if we progress along a CV, we should be able to access a contiguous configuration from any other configuration
  - **Continuously differentiable:** transition between one and another configuration is smooth

# Collective variable

- Why collective variable are useful to restrain our system?
  - Let's define a collective variable ξ that is such that $\xi = 0$ at the center of geometry of the "blue" atoms between $x + dx$ and $y + dy$

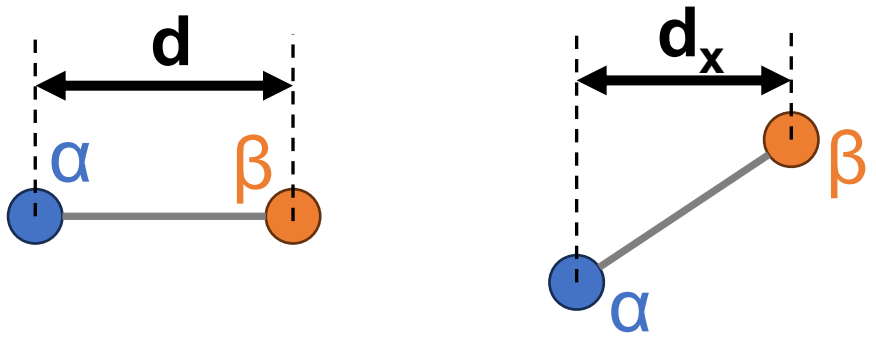



  - Along CVs, we can apply the same biasing potential, but we can control how they are applied and their effect finely

# GROMACS pulling code

- Enabling the pulling code
  - In the mdp:

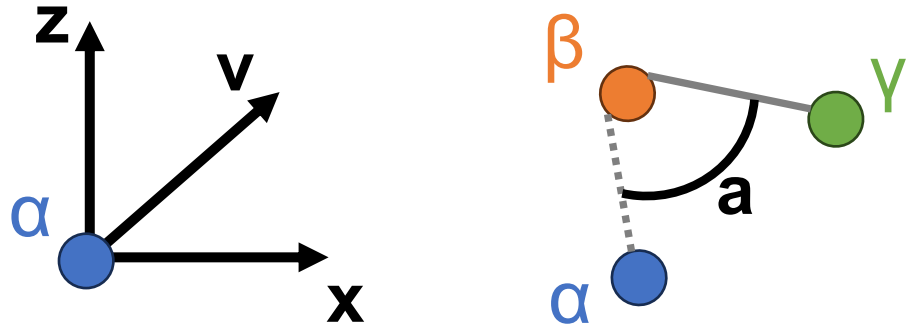
```
pull = yes
```



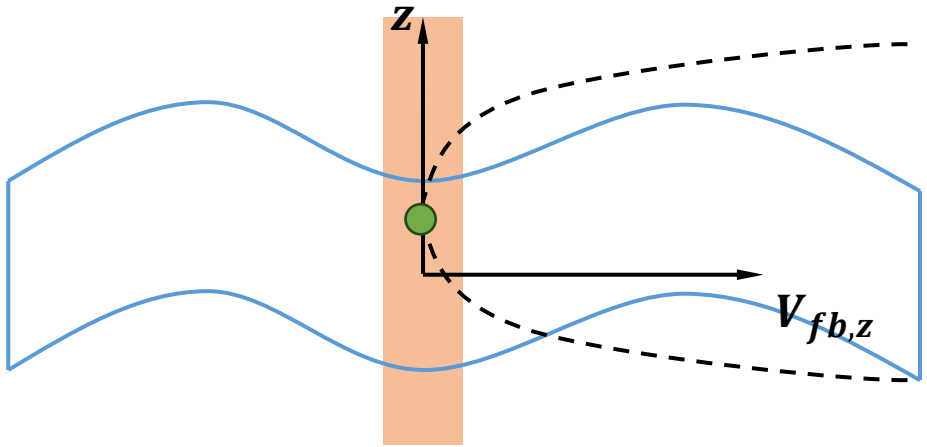

```
pull-coord1-dim = Y N N
```

- Defining the CVs
  - CVs in GROMACS are called geometries:

```
pull-ncoords = 4
pull-coord1-geometry = distance
pull-coord2-geometry = direction
pull-coord3-geometry = angle
pull-coord4-geometry = cylinder
```




```
pull-coord2-vec = 3.6 0 3.6
```




```
pull-cylinder-r = 2.2
```

- There are more geometries available, see the documentation!

# GROMACS pulling code

- Defining the pulling groups
  - Define the groups (center of mass)

```
pull-ngroups = 3
pull-group1-name = Water
pull-group1-name = Membrane
pull-group1-name = Ions
```

Defined in the index file

- Defining the frequency at which output are saved
  - First output – value of CVs:

```
pull-nstxout = 1000
```

Saving once every 1000 step

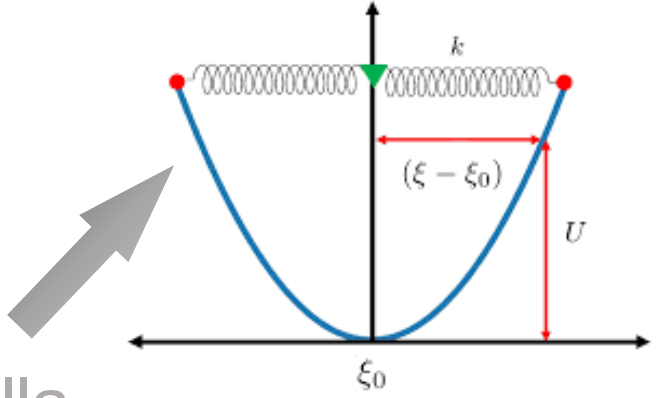
  - Second output – value of the instantaneous force a applied to a pulled group by a biasing potential

```
pull-nstfout = 1000
```

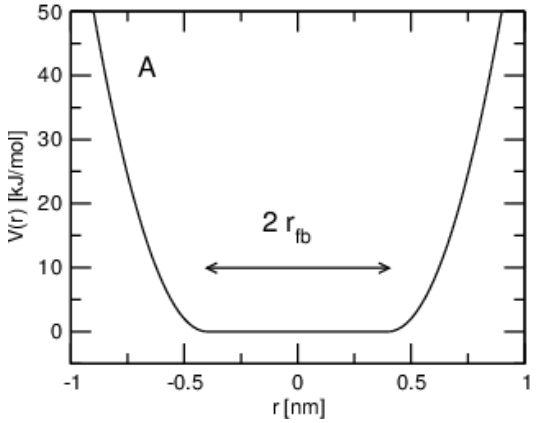
# GROMACS pulling code

- Defining the type of biasing potential
  - Define the groups (center of mass)

```
pull-coord1-type = umbrella
pull-coord2-type = flat-bottom
pull-coord3-type = flat-bottom-high
```




For umbrella potentials



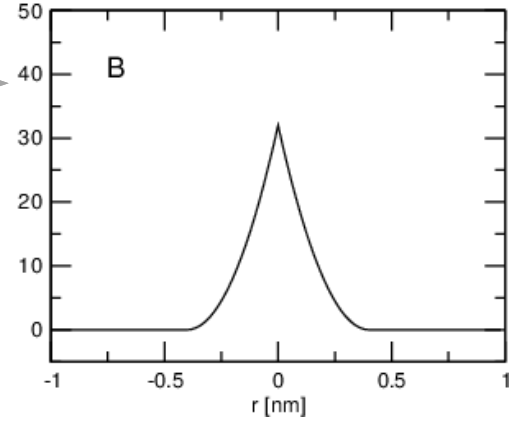

- Parameters of the biasing potential
  - Starting position of the center of the biasing potential $\xi_0$

```
pull-coord1-init = 1.2
```

  - Size of the flat area $r_{fb}$

```
pull-coord1-init = 0.5
```

For flat-bottom potentials




  - Force constant $k$

```
pull-coord1-k = 1000
```

  - Pulling rate [ nm/ps ] or [ deg/ps ]

```
pull-coord1-rate = 0.001
```

# QUESTION TIME